

VIRGO

Building a Small Language Model from Scratch

“Not Everybody Needs The Blueprint.”

*A first-year student's honest account of teaching a machine to talk —
from raw text files to a model that can hold a conversation.*

Punit Kumar Kashyap

First-Year Undergraduate Student
Department of Engineering Physics
Indian Institute of Technology Dharwad

Project: VIRGO Small Language Model

Phases Covered: I — Virgo Base · II — Virgo Chat · III — Virgo Instruct

Model Size: ~119.6M Parameters

Report Type: Personal Learning Project / Technical Report

July 2026

Abstract

VIRGO is a decoder-only Small Language Model (SLM) that I built completely on my own, as a first-year engineering student, mostly out of curiosity and stubbornness. The goal was never to make something that beats ChatGPT or Gemini — it was to actually understand, with my own hands and my own bugs, how models like ChatGPT, Gemini, and Claude work under the hood.

This meant going through the whole pipeline myself: collecting and cleaning a huge pile of text, training my own tokenizer, writing the Transformer architecture, pretraining it on raw text so it could learn language, and then teaching it to chat and follow instructions. VIRGO went through three phases — Virgo Base, Virgo Chat, and Virgo Instruct — each one building on the last.

VIRGO is tiny compared to the language models running in production today, and it has real limitations that I've been upfront about in this report. But it follows the same basic recipe those bigger models use, and building it taught me more about language models in a few months than any article or lecture could have.

Keywords: Small Language Models, Transformer Architecture, Decoder-only Models, Byte-Level BPE Tokenization, Rotary Position Embeddings, Instruction Fine-Tuning, Manually Curated Datasets, Language Model Pretraining.

1. Why I Started This Project

It started with a pretty simple question that I couldn't stop thinking about:

How do models like ChatGPT, Gemini, and Claude actually understand language and reply like a person would?

As a first-year student, most of what I knew about large language models came from YouTube lectures, blog posts, and research papers that I could only half-follow. Reading about Transformers in theory is one thing. I wanted to know if I could actually build one myself — even a tiny one — and watch it slowly learn to form real sentences instead of gibberish.

So instead of just reading more, I decided to build a Transformer-based language model completely from scratch and go through every single stage myself: collecting data, training a tokenizer, writing the model architecture, pretraining it, and finally teaching it to actually hold a conversation and follow instructions.

VIRGO is nowhere near the scale of real production systems, but it follows the exact same workflow: dataset preparation, tokenizer training, pretraining, fine-tuning, instruction tuning, and inference. For me this project was never really about the final model — it was about the process of learning by building something that either works or breaks in front of you, with no one to blame but your own code.

1.1 Why the Name “VIRGO”?

The project is named VIRGO after my zodiac sign — a small personal touch on my first big AI project. But more than that, the name marks the start of a longer journey. VIRGO is meant to be the first model in a family, and the foundation for bigger, more capable versions I hope to build later in my degree.

2. Project Overview

VIRGO was built in three phases, each one solving a different problem and building directly on top of the last phase's checkpoint.

Phase	Objective
Phase I — Virgo Base	Pretrain the base language model on raw, general-domain text.
Phase II — Virgo Chat	Fine-tune Virgo Base so it can hold a conversation.
Phase III — Virgo Instruct	Fine-tune further on a hand-built dataset so it reliably follows instructions and generalizes better.

This phased approach let me improve the model gradually instead of trying to solve everything at once — language first, then conversation, then instruction-following. Figure 1 shows the three phases side by side.

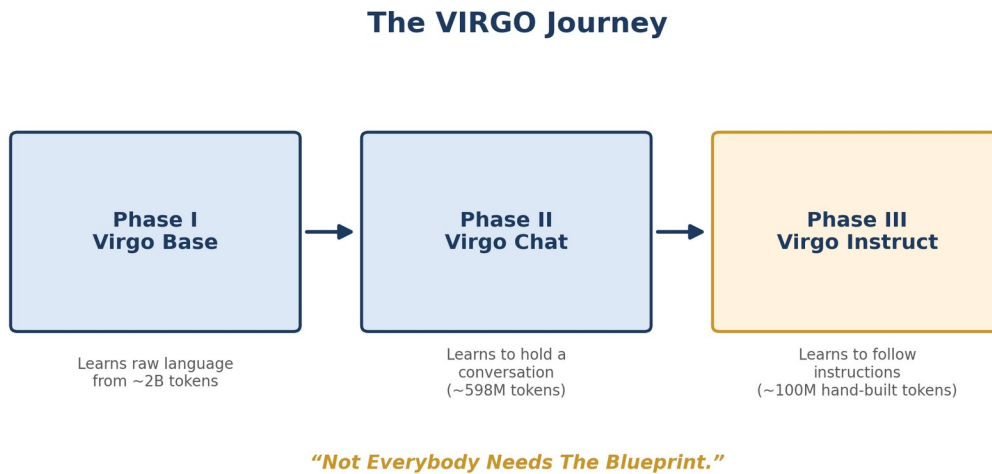


Figure 1. The three-phase VIRGO journey — from raw language to an instruction-following assistant.

3. Phase I — Virgo Base

3.1 Objective

Virgo Base is a decoder-only SLM trained completely from scratch on roughly 2 billion tokens. Unlike a chat assistant, Virgo Base's only job in this phase was to learn the basic shape of language through self-supervised causal language modeling — predicting the next token given everything that came before it.

Through this training, the model gradually picked up:

- Grammar and sentence structure
- Vocabulary and word usage
- Syntax and common linguistic patterns
- Sentence-level coherence
- Semantic relationships between words
- Contextual understanding
- Basic factual knowledge picked up from the text itself
- Long-range dependencies across a passage
- The core skill everything else is built on: next-token prediction

This phase is really the foundation everything else in VIRGO stands on.

3.2 Dataset Preparation

I collected around 2 billion high-quality tokens from a mix of educational and general-domain sources, trying to cover as many topics as possible so the model wouldn't just learn one narrow style of writing. The corpus pulled from subjects including Computer Science, Mathematics, Physics, Chemistry, Biology, Engineering, History, Economics, Social Sciences, Philosophy, Literature, and General Knowledge.

Before any of it touched the model, every document went through cleaning, normalization, deduplication, language filtering, category balancing, and quality checks. This step mattered more than I expected — raw scraped text is full of duplicated passages, broken formatting, and junk that quietly wrecks training if you don't catch it early.

3.3 Tokenizer

Type: Byte-Level Byte Pair Encoding (Byte-Level BPE) • Vocabulary Size: 45,000 tokens

Special tokens: <pad>, <unk>, <bos>, <eos>, <newline>, <tab>

Instead of reusing someone else's tokenizer, I trained a Byte-Level BPE tokenizer from scratch on VIRGO's own corpus. Training my own tokenizer meant the vocabulary actually matched the statistical patterns of the text VIRGO would be trained on, which gave more efficient subword representations and far fewer unknown tokens.

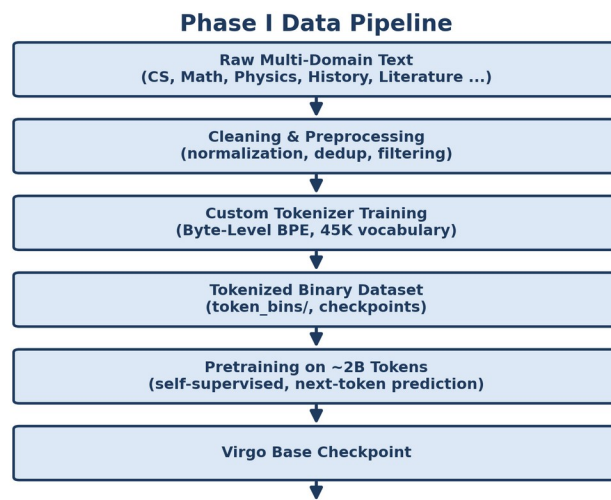


Figure 2. The Phase I data pipeline, from raw text to a pretrained Virgo Base checkpoint.

4. Virgo Base Architecture

Virgo Base follows a decoder-only Transformer architecture — the same family of architecture used by modern autoregressive language models. It's a stack of Transformer decoder blocks, each processing token sequences through masked self-attention.

4.1 Token Embedding Layer

Every token in the 45K vocabulary gets mapped to a learned vector in a 768-dimensional space, which is then refined layer by layer as it moves through the network.

4.2 Rotary Position Embeddings (RoPE)

Instead of learned absolute position embeddings, VIRGO uses Rotary Position Embeddings (RoPE), which rotate the query and key vectors based on token position. This lets the model reason about relative distance between tokens rather than their absolute position, which helps it generalize better to longer sequences without adding extra learned parameters.

4.3 Multi-Head Self-Attention

Each token can only attend to tokens that came before it, enforced through a causal mask. Multiple attention heads run in parallel, letting the model learn different kinds of patterns at once — syntax, entity references, and longer contextual dependencies.

4.4 Feed-Forward Network (MLP)

A position-wise feed-forward network adds non-linearity on top of attention. In Virgo Base, this expands the 768-dimensional hidden state up to 3072 dimensions and projects it back down, following the standard 4x expansion ratio used in most Transformer decoders.

4.5 Layer Normalization & Residual Connections

Layer normalization keeps activations stable during training, which really does make convergence smoother. Residual connections give gradients a direct path around each sub-layer, which is what makes stacking 12 layers deep actually trainable instead of falling apart.

4.6 Linear Output Projection

The final hidden states get projected back into vocabulary space through a linear layer, producing a probability distribution over all 45K tokens for next-token prediction.

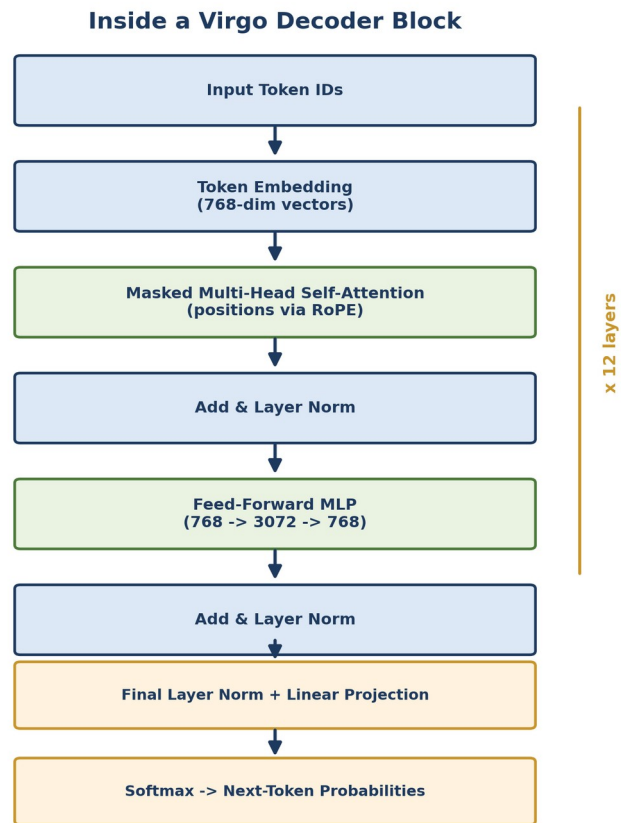


Figure 3. One Virgo decoder block, repeated 12 times to form Virgo Base.

4.7 Model Configuration

Parameter	Value
Model Type	Decoder-only Transformer
Vocabulary Size	45K
Transformer Layers	12
Hidden Dimension	768
Attention Heads	12
Feed-Forward Dimension	3072
Maximum Context Length	1024 tokens
Dropout	0.1
Total Parameters	~119.6M

4.8 Phase I Outputs

Data preparation artifacts: `virgo_tokenizer.json`, `token_bins/`, `raw_categories/`, `tokenizer_training_data/`, `dataset_checkpoint.json`, `virgo_progress.zip`. The pipeline supports streaming datasets, checkpoint recovery, binary serialization, and incremental preprocessing so I could pause and resume without redoing expensive steps.

Training artifacts: model checkpoints, optimizer state, LR scheduler state, validation metrics, loss history, and the final pretrained Virgo Base model — which became the starting point for Phase II.

5. Phase II — Virgo Chat

5.1 Objective

Phase II turns Virgo Base into Virgo Chat through instruction tuning on around 598 million conversational tokens. Instead of learning language from raw text like in Phase I, Virgo Chat was trained to:

- Follow instructions
- Understand conversational context
- Maintain consistency across a dialogue
- Develop a consistent assistant identity
- Produce safe, coherent responses
- Improve overall response quality

...all while keeping the language understanding it picked up during Phase I.

5.2 Chat Dataset

Dataset	Conversations
OpenHermes 2.5	500K
OpenOrca	400K
UltraChat 200K	400K
WildChat	300K
UltraFeedback	200K
Magpie	200K
Virgo Identity Dataset	Repeated 20x

Every dataset was converted into a unified System → User → Assistant format so the structure stayed consistent no matter where the data came from. The preprocessing pipeline handled HTML cleaning, unicode normalization, assistant-identity replacement (so it responds as “Virgo”), duplicate removal via SHA-256 hashing, quality scoring, conversation filtering, and packing everything into fixed 1024-token sequences. The final dataset was split 99% train / 1% validation.

5.3 Phase II Outputs

Data artifacts: virgo_chat_dataset.csv, virgo_chat_dataset.zip, chat_train.bin, chat_val.bin, train.bin, val.bin, checkpoint.json.

Training artifacts: virgo_chat_best.pt, virgo_chat_last.pt, training_state.pt, validation metrics, sample generated responses, and the fine-tuned Virgo Chat model.

6. Phase III — Virgo Instruct

6.1 Objective

Phase III is the part of this project I'm honestly most proud of. Virgo Chat could hold a conversation, but it still wasn't great at reliably obeying an instruction, staying on-task, or generalizing to a phrasing of a request it hadn't seen before. So the goal of Phase III was to fine-tune Virgo Chat into Virgo Instruct — a version that actually listens to what the user asks for and generalizes across many different kinds of instructions, not just conversational chit-chat.

6.2 A Dataset Built by Hand

Unlike Phase I and Phase II, which leaned on existing open-source datasets, the Phase III dataset was something I built myself, by hand, from scratch. There was no ready-made corpus for what I needed — so I sat down and manually wrote instruction-and-response pairs, one at a time, covering a wide range of tasks, tones, and phrasings so the model wouldn't just memorize a narrow pattern.

This dataset ended up at roughly 100 million tokens, and every single one of them passed through my own hands — writing, checking, rewriting, and formatting. It took several hours across many sessions, and there were plenty of moments where it felt tedious and slow, but this was also the part of the project where I understood, very concretely, why data quality is treated as such a big deal in real language model teams. A messy, inconsistent dataset here would have undone everything Phase I and II had built.

Phase III: Building the Instruction Dataset by Hand



Hours of manual writing, one conversation at a time.

Figure 4. Manually building the Phase III instruction dataset — write, review, clean, repeat.

6.3 What Phase III Actually Changed

After fine-tuning on this hand-built dataset, Virgo Instruct showed a real, noticeable improvement in:

- Following explicit instructions instead of drifting off-topic
- Generalizing to instruction phrasings it hadn't seen during training
- Staying consistent in tone and format across a response
- Handling a wider variety of task types with a single model

It's still a small model with small-model limits, but Phase III is what pushed VIRGO from “can chat a bit” to “can actually be told what to do and attempt it properly.”

7. Limitations

I want to be honest about what VIRGO can't do, instead of only talking about what it can. Being upfront about this feels more important to me than making the report sound impressive. As of this report, VIRGO does not support:

- Web search — it has no access to live information and only knows what was in its training data.
- Image generation — it is a text-only model with no visual generation capability.
- High-level reasoning — multi-step logical reasoning is inconsistent at this scale.
- Mathematics solving — it struggles with anything beyond very basic arithmetic or symbolic manipulation.
- File creation — it cannot generate or export documents, spreadsheets, or other files.
- Longer contextual chats — the maximum context length is only 1024 tokens, so long conversations get cut off or lose earlier context.
- Memory — it has no persistent memory across sessions or turns beyond what fits in a single context window.
- Continuous conversation — currently VIRGO is a single-turn prompt-to-response model rather than a model that tracks an ongoing multi-turn dialogue state.

These aren't things I'm hiding or glossing over — they're a direct result of the model's small size (~119.6M parameters), the limited training compute available to a first-year student working mostly alone, and the scope of what a first version of this project could reasonably cover.

8. Engineering Highlights

A few engineering practices that made this project actually possible to finish as one person, across both the data pipeline and the training pipeline:

- Streaming dataset processing
- Deterministic, resumable preprocessing pipeline
- Memory-mapped binary datasets
- Custom Byte-Level BPE tokenizer
- RoPE-based Transformer architecture
- Mixed-precision training
- Automatic checkpoint recovery
- Packed token sequences
- A hand-built, quality-checked instruction dataset (Phase III)
- Scalable fine-tuning workflow
- An interactive inference framework for talking to the model directly

9. Conclusion & What's Next

Building VIRGO end-to-end — from raw text files to a model that can actually follow an instruction and hold a conversation — taught me far more than reading about Transformers ever could. Every stage came with its own problems: cleaning messy data, getting the tokenizer to behave, debugging attention, and staring at loss curves that refused to go down until I found the one small bug breaking everything.

As a first-year student, I don't think of VIRGO as a finished product. It's the first real milestone of a much longer journey. Looking ahead, there's a clear path I'd like to take this project on:

- Scaling VIRGO up from a Small Language Model into a genuinely Large Language Model, with more parameters, more data, and more compute.
- Extending the context window well beyond 1024 tokens so it can hold longer, more coherent conversations.
- Adding real multi-turn memory so it stops being purely prompt-to-response.
- Improving mathematical and logical reasoning through targeted fine-tuning and better data.
- Exploring retrieval and tool-use, so the model can eventually pull in outside information instead of relying only on what it memorized.
- Experimenting with multimodal capability — letting VIRGO understand images, not just text.
- Building an actual evaluation suite instead of judging quality by eye, so future versions can be compared properly.

For now, VIRGO stands as proof that a curious first-year student, with enough patience and no shortage of failed training runs, can build a working language model from scratch — and that this is only the beginning of where I want to take it.

A Personal Note from the Author

I want to end this report the way I started this project — honestly.

VIRGO was built entirely on my own. No lab, no mentor sitting beside me, no team, no external funding — just a laptop, a lot of tutorials, a lot of documentation, and an unreasonable number of late nights spent staring at a loss curve that wouldn't move. There were moments I genuinely wanted to give up, especially during Phase III, sitting alone writing instruction pairs by hand, one after another, for hours, wondering if any of it would actually matter in the end.

But somewhere along the way I realized something that became the whole spirit of this project: not everybody needs the blueprint. Nobody handed me a step-by-step manual for building a language model as a first-year student. I didn't have a clean, guaranteed path. I just had a question I couldn't let go of, and a willingness to fail at something a hundred times until it finally worked once.

| *Not Everybody Needs The Blueprint.*

If you're reading this as another student wondering whether you're “ready” to build something like this — you probably are more ready than you think. You don't need permission, a perfect roadmap, or someone else's blueprint. You need curiosity, patience with your own failures, and the willingness to start before you feel prepared.

This project is dedicated to that idea. VIRGO is small, imperfect, and has plenty of limitations — but it is entirely mine, built with my own hands, my own mistakes, and my own late nights. And that, more than any benchmark score, is what I'm proudest of.

— Punit Kumar Kashyap

First-Year Undergraduate, Engineering Physics, IIT Dharwad

— End of Report —